

Advanced Physics in Unity

Cách để gameplay code giao tiếp trực tiếp với thế giới vật lý

Ý TƯỞNG CỐT LÕI

Physics Simulation vs Physics Query

Physics Simulation: Unity tự động tính toán trọng lực, rơi, va đập và chuyển động dựa vào Rigidbodies.

Physics Query: Code của chúng ta chủ động hỏi: "*Có bức tường nào trước mặt không?*" hoặc "*Ai vừa bước vào phạm vi vụ nổ?*"

4 Công Cụ Truy Vấn Vật Lý Chính

Công cụ (Tool)	Câu hỏi đặt ra (Query Question)	Trường hợp áp dụng phổ biến (Use Case)
<u>Raycast</u>	" <u>Có vật thể nào dọc theo đường thẳng này không?</u> "	<u>Cơ chế bắn súng</u> , tương tác nút bấm, <u>kiểm tra mặt đất</u> .
<u>Trigger</u>	"Có thực thể nào vừa bước vào/ra khỏi vùng này?"	<u>Nhặt vật phẩm</u> , <u>checkpoint</u> , vùng bẫy gây sát thương.
<u>Collision</u>	"Có sự chạm mạnh vật lý giữa hai thực thể cứng?"	<u>Vật thể nảy lại</u> , phát âm thanh va chạm, tính sát thương.
<u>Overlap</u>	"Có những vật gì đang nằm trong khu vực này ngay bây giờ?"	Tầm sát thương vụ nổ, <u>nhát chém kiếm</u> , quét địch lân cận.

Tool 1: Raycast (Bắn Tia Laser)

Kiểm tra theo đường thẳng tuyến tính

Bắn một tia Laser vô hình từ một điểm bắt đầu theo một hướng chỉ định. Tia này sẽ trả về thông tin nếu nó chạm vào Collider bất kỳ.

- Origin & Direction: Điểm xuất phát và hướng bắn của tia laser.
- LayerMask: Bộ lọc giúp tia laser bỏ qua các lớp không cần thiết (Ví dụ: bỏ qua mây hoặc cỏ cỏ).
- **Lưu ý quan trọng**: Nếu Origin nằm TRONG collider, Raycast sẽ KHÔNG nhận diện được collider đó.



SimpleRaycast

Quy tắc hiệu năng

Đừng quét tất cả mọi thứ! Luôn ưu tiên dùng LayerMask để khoanh vùng mục tiêu cần dò tìm (như Địch hoặc Tường).

Tăng tốc xử lý CPU:

`Physics.Raycast(..., _hitMask);`

```
using UnityEngine;
```

```
public class SimpleRaycast : MonoBehaviour
```

```
{
```

```
    [SerializeField] private float _distance = 10f; ✓
```

```
    [SerializeField] private LayerMask _hitMask; ✓
```

```
    private void Update()
```

```
    {
```

```
        Vector3 origin = transform.position; Vector3 direction =  
transform.forward;
```

```
        if (Physics.Raycast(origin, direction, out RaycastHit hit, _distance,  
_hitMask))
```

```
        {
```

```
            Debug.DrawRay(origin, direction * hit.distance, Color.red); ✓
```

```
            Debug.Log($"Hit: {hit.collider.name}");
```

```
        }
```

```
    else
```

```
    {
```

```
        Debug.DrawRay(origin, direction * _distance, Color.green); ✓
```

```
    }
```

```
    }
```

```
}
```

| Raycast Để Tương Tác Vật Thể

Góc nhìn thứ nhất (FPP)

Sử dụng tâm camera bắn một tia về phía trước để kích hoạt cơ chế bấm nút bấm, mở cửa khi người chơi bấm nút "E".

Tại sao dùng Interface?

Thay vì so sánh tên của đối tượng (ví dụ: name == "Door"), chúng ta chỉ cần hỏi xem đối tượng có chứa Interface `IInteractable` hay không.

```
using UnityEngine;

public interface IInteractable
{
    void Interact();
}

public class CameraInteractor : MonoBehaviour
{
    [SerializeField] private Camera _camera;
    [SerializeField] private float _interactDistance = 3f;
    [SerializeField] private LayerMask _interactableMask;
    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.E)) TryInteract();
    }
    private void TryInteract()
    {
        Ray ray = new Ray(_camera.transform.position,
            _camera.transform.forward);
        if (Physics.Raycast(ray, out RaycastHit hit, _interactDistance,
            _interactableMask))
        {
            if (hit.collider.TryGetComponent(out IInteractable interactable))
            {
                interactable.Interact();
            }
        }
    }
}
```

Tool 2: Trigger (Vùng Cảm Biến)

Vùng đi xuyên qua được

Trigger không cản chuyển động vật lý. Chúng là vùng cảm biến vô hình, chạy code khi có vật thể chạm vào hoặc rời đi.

Yêu cầu kỹ thuật:

- Một trong hai vật chạm nhau PHẢI có thành phần **Rigidbody**.
- Đã tick bật lựa chọn **Is Trigger** trên Collider.

```
using UnityEngine;

public class CoinPickup : MonoBehaviour
{
    [SerializeField] private int _value = 1;

    private void OnTriggerEnter(Collider other)
    {
        if (!other.CompareTag("Player")) return;
        Debug.Log($"Collected coin worth {_value}");
        Destroy(gameObject);
    }
}
```

Enter / Stay / Exit

1 ∞ 1

Trigger Để Quản Lý Tầm Phát Hiện

Hoạt động theo Event

Sử dụng OnTriggerEnter và OnTriggerExit để lưu vết và hủy vết Player khi nằm trong tầm quét tầm gần của quái.

Lợi thế: Cơ chế này giúp tối ưu CPU vì chỉ chạy code khi xảy ra hành động di chuyển ra/vào khu vực, thay vì quét từng Frame trong hàm Update.

```
using UnityEngine;

public class EnemyDetectionZone : MonoBehaviour
{
    private Transform _target;
    public bool HasTarget => _target != null;

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            _target = other.transform;
            Debug.Log("Player entered range");
        }
    }

    private void OnTriggerExit(Collider other)
    {
        if (other.CompareTag("Player") && other.transform == _target)
        {
            _target = null;
            Debug.Log("Player left range");
        }
    }
}
```


Tool 3: Collision (Va Chạm Vật Lý)

Tương tác va đập cứng

Khác với Trigger, hai đối tượng va chạm vật lý cứng sẽ tự động cản đường nhau, tạo góc nảy hoặc dừng đứng lại.

Dữ liệu va chạm chi tiết

Thông tin trả về chứa các điểm tiếp xúc (Contact Points) và vận tốc phản hồi lực tương đối (Relative Velocity).

```
using UnityEngine;
```

```
public class CollisionLogger : MonoBehaviour
```

```
{
```

```
    private void OnCollisionEnter(Collision collision)
```

```
    {
```

```
        Debug.Log($"Collided with: {collision.gameObject.name}");
```

```
        Debug.Log($"Impact speed: {collision.relativeVelocity.magnitude}");
```

```
    }
```

```
}
```

Collision: Sát Thương Theo Lực Đập

Đo lường lực va chạm

Sử dụng `relativeVelocity.magnitude` để đo tốc độ đâm. Nếu đâm nhẹ, bỏ qua. Nếu đâm cực mạnh, tính toán sát thương trừ vào máu (Health).

Hệ thống máu tách biệt chạy dưới dạng một Class độc lập giúp tái sử dụng linh hoạt.

```
using UnityEngine;

public class ImpactDamage : MonoBehaviour
{
    [SerializeField] private float _minimumSpeed = 5f;
    [SerializeField] private int _damage = 10;

    private void OnCollisionEnter(Collision collision)
    {
        float speed = collision.relativeVelocity.magnitude;

        if (speed < _minimumSpeed) return;

        if (collision.gameObject.TryGetComponent(out Health health))
        {
            health.TakeDamage(_damage);
            Debug.Log($"Impact damage dealt. Speed = {speed}");
        }
    }
}
```

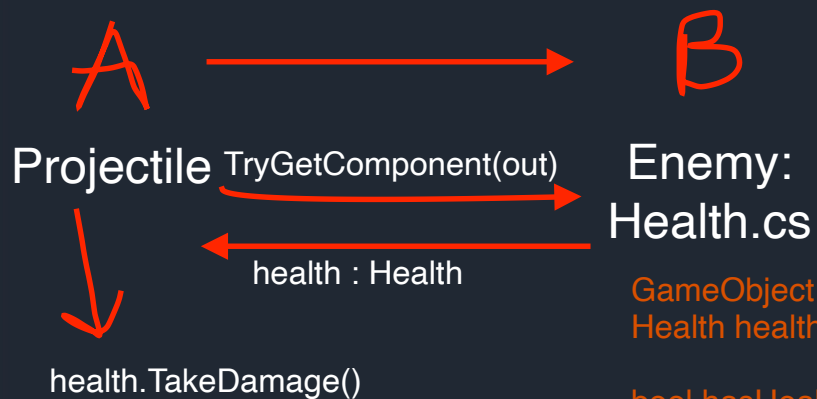
Collision Cho Projectile (Đạn Bay)

Khởi tạo hiệu ứng tia lửa

Lấy điểm tiếp xúc đầu tiên (contacts[0]) để sinh ra Spark/Hit Effect hướng trực tiếp ra ngoài bề mặt tiếp xúc va chạm.

Lưu ý về đạn:

Nếu đạn bay tốc độ cực nhanh, kỹ thuật Raycast sẽ tối ưu và đỡ lỗi hơn kỹ thuật dùng Collision thực thể.



```
GameObject go = new GameObject();  
Health health = go.AddComponent<Health>();  
  
bool hasHealth = collision.gameObject.TryGetComponent<Health>();  
if (hasHealth)  
{  
    health = collision.gameObject.GetComponent<Health>();  
    health.TakeDamage(_damage);  
}
```

```
using UnityEngine;
```

```
public class Projectile : MonoBehaviour
```

```
{
```

```
[SerializeField] private int _damage = 25; ✓
```

```
[SerializeField] private GameObject _hitEffectPrefab; ✓
```

```
private void OnCollisionEnter(Collision collision)
```

```
{
```

```
if (collision.gameObject.TryGetComponent(out Health health))
```

```
{
```

```
    health.TakeDamage(_damage);
```

```
}
```

```
    ContactPoint contact = collision.contacts[0]; ✓
```

```
if (_hitEffectPrefab != null)
```

```
{
```

```
    Instantiate(_hitEffectPrefab, contact.point,  
    Quaternion.LookRotation(contact.normal)); ✓
```

```
}
```

```
    Destroy(gameObject); ✓
```

Tool 4: Overlap (Quét Vùng)

Truy vấn diện tích tức thời

Không dùng để theo dõi liên tục. Overlap dùng để quét và "chụp" lấy danh sách Collider hiện diện tại thời điểm kích hoạt.

Sát thương nổ lan

Sử dụng Physics.OverlapSphere lấy mảng Collider để áp dụng trừ máu hàng loạt cho quái xung quanh.

```
using UnityEngine;

public class Explosion : MonoBehaviour
{
    [SerializeField] private float _radius = 5f;
    [SerializeField] private int _damage = 40;
    [SerializeField] private LayerMask _damageMask;

    public void Explode()
    {
        Collider[] hits = Physics.OverlapSphere(transform.position, _radius,
            _damageMask, QueryTriggerInteraction.Ignore);

        foreach (Collider hit in hits)
        {
            if (hit.TryGetComponent(out Health health))
            {
                health.TakeDamage(_damage);
            }
        }

        Destroy(gameObject);
    }
}
```

OverlapBox: Quét Tầm Chém Của Kiếm

Tia Laser quá mỏng?

Dùng Raycast cho lưỡi kiếm rất dễ chém hụt do chuyển động siêu tốc của kiếm. Giải pháp là dùng một chiếc hộp OverlapBox mô phỏng diện tích lưỡi kiếm chém ra.

Quét hình hộp theo vị trí và góc quay thực của khớp tay hoặc lưỡi kiếm.

```
using UnityEngine;
```

```
public class MeleeAttack : MonoBehaviour
```

```
{
```

```
    [SerializeField] private Transform _attackCenter;
```

```
    [SerializeField] private Vector3 _halfExtents = Vector3.one;
```

```
    [SerializeField] private int _damage = 20;
```

```
    [SerializeField] private LayerMask _enemyMask;
```

```
    public void Attack()
```

```
{
```

```
        Collider[] hits = Physics.OverlapBox( _attackCenter.position,  
        _halfExtents, _attackCenter.rotation, _enemyMask,  
        QueryTriggerInteraction.Ignore );
```

```
        for (int i = 0; i < hits.Length; i++)
```

```
{
```

```
            if (hits[i].TryGetComponent(out Health health))
```

```
{
```

```
                health.TakeDamage(_damage);
```

```
}
```

```
}
```

```
}
```

```
}
```

Tối Ưu Overlap Không Sinh Bộ Nhớ Rác

Vấn đề phân bổ mảng mới

OverlapSphere thông thường tạo ra một mảng mới hoàn toàn sau mỗi lần gọi, gây gánh nặng dọn dẹp bộ nhớ (Garbage Collection).

Giải pháp tối ưu

Khởi tạo sẵn một mảng cố định một lần, sau đó ghi đè giá trị liên tục qua hàm có đuôi **NonAlloc**.

```
using UnityEngine;
```

```
public class EnemyScanner : MonoBehaviour
```

```
{
```

```
    [SerializeField] private float _scanRadius = 10f;
```

```
    [SerializeField] private LayerMask _enemyMask;
```

```
    private readonly Collider[] _results = new Collider[20];
```

```
    private void Update()
```

```
{
```

```
        int count = Physics.OverlapSphereNonAlloc( transform.position,  
            _scanRadius, _results, _enemyMask, QueryTriggerInteraction.Ignore );
```

```
        for (int i = 0; i < count; i++)
```

```
{
```

```
            Debug.Log($"Detected enemy: {_results[i].name}");
```

```
}
```

```
}
```

```
}
```

| Phối Hợp Trigger & Raycast Cho Enemy Vision

Chiến thuật phối hợp

Sử dụng Trigger quét bán kính trước (nhẹ CPU). Khi phát hiện Player ở trong vòng quét, lập tức bắn Raycast để kiểm tra xem có bức tường nào chắn giữa hay không.

Nhờ vậy quái vật sẽ không thể nhìn xuyên qua tường được.

```
using UnityEngine;
```

```
public class EnemyVision : MonoBehaviour
```

```
{
```

```
    [SerializeField] private Transform _eye;
```

```
    [SerializeField] private LayerMask _obstacleMask;
```

```
    private Transform _target;
```

```
    private void OnTriggerEnter(Collider other)
```

```
{
```

```
        if (other.CompareTag("Player")) _target = other.transform;
```

```
}
```

```
    private void OnTriggerExit(Collider other)
```

```
{
```

```
        if (other.transform == _target) _target = null;
```

```
}
```

```
    private void Update()
```

```
{
```

```
        if (_target == null) return;
```

```
        Vector3 dir = (_target.position - _eye.position).normalized;
```

```
        float dist = Vector3.Distance(_eye.position, _target.position);
```

```
        bool blocked = Physics.Raycast(_eye.position, dir, dist,
```

```
        _obstacleMask);
```

```
        if (!blocked) Debug.Log("Enemy SEES Player!");
```

```
}
```

```
}
```

Các Lỗi Hay Gặp Trong Unity Physics

- ⚠️ **Trigger không hoạt động:** Đảm bảo đã tick chọn "Is Trigger" và có ít nhất 1 Rigidbody đính kèm ở 1 trong 2 vật thể va chạm.
- ⚠️ **Collision không chạy:** Đảm bảo "Is Trigger" đã được tắt hoàn toàn và có ít nhất 1 Rigidbody không kích hoạt cơ chế "Is Kinematic" khi va đập.
- ⚠️ **Raycast không bắn trúng vật thể:** Raycast sẽ hỏng hoàn toàn nếu điểm xuất phát (Origin) nằm kẹt ngay trong một Collider khác. Hãy dịch chuyển Origin ra phía trước một chút.
- ⚠️ **Bị tụt FPS nghiêm trọng:** Tránh lạm dụng các hàm sinh mảng liên tục như `OverlapSphere` hay `OverlapBox` trực tiếp trong `Update`. Hãy chuyển sang các hàm tối ưu có đuôi `NonAlloc`.

Demo

Q&A